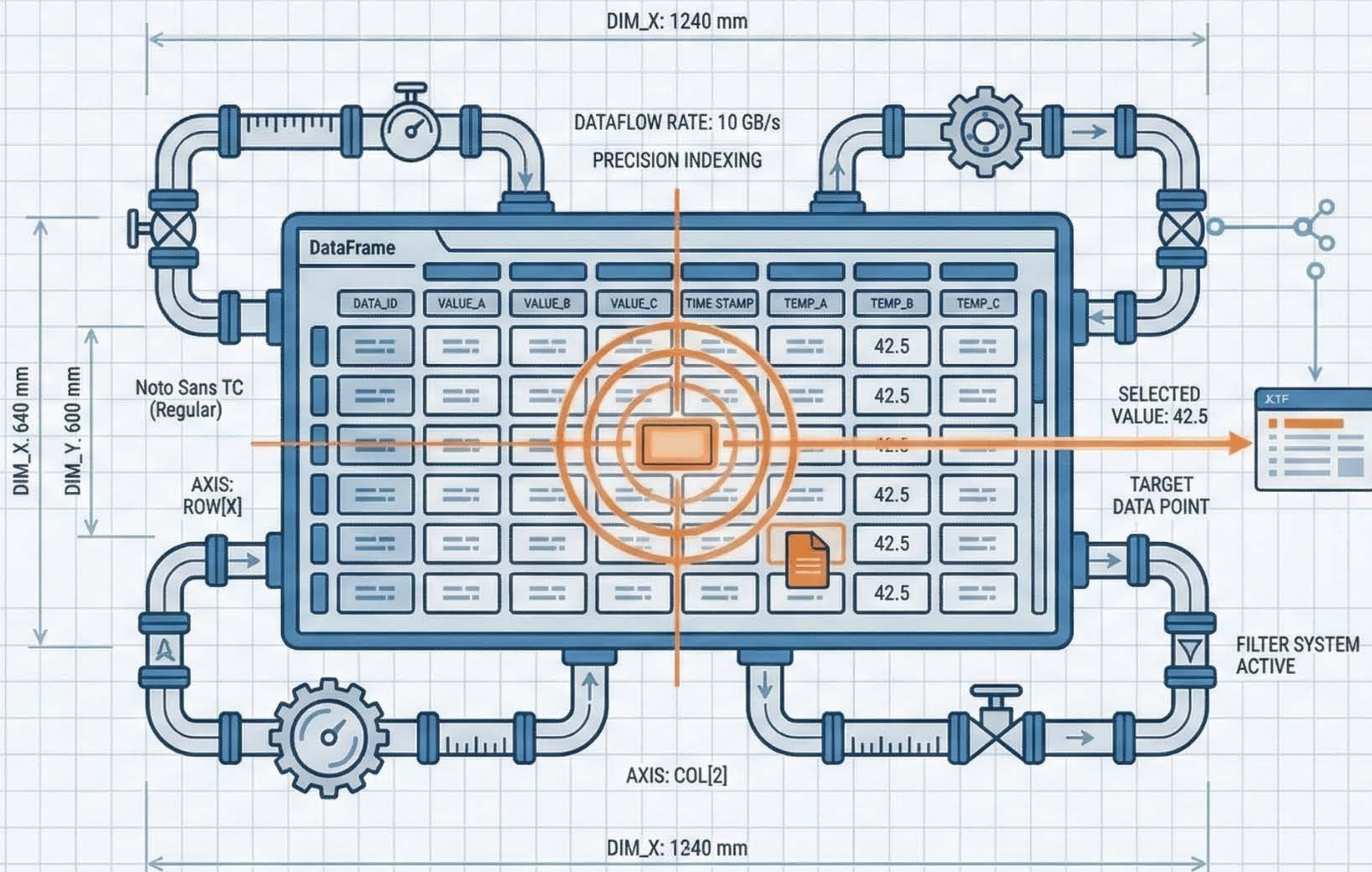
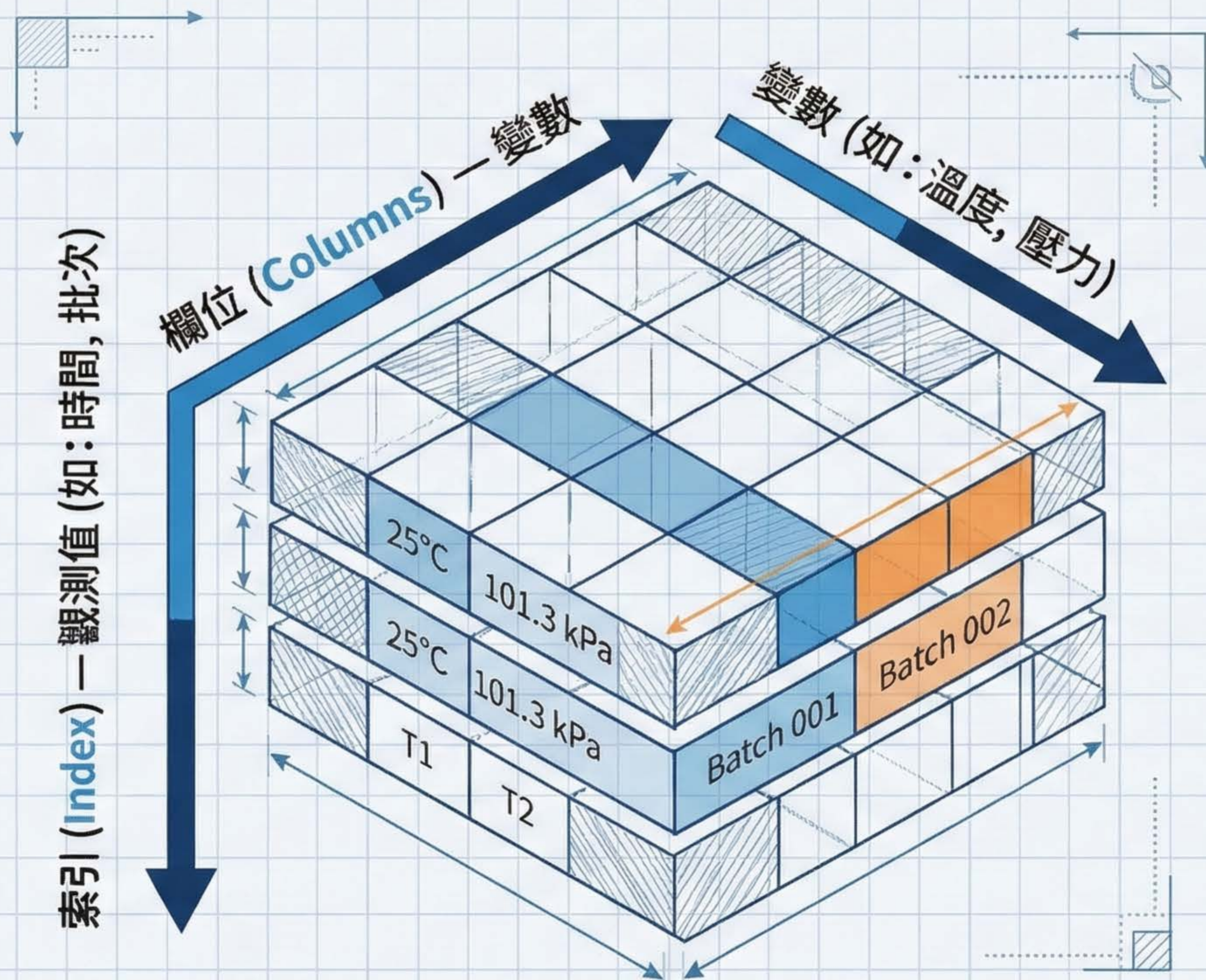


Pandas 資料選取與索引

精準提取關鍵數據的導航系統



數據的座標系統：二維網格結構

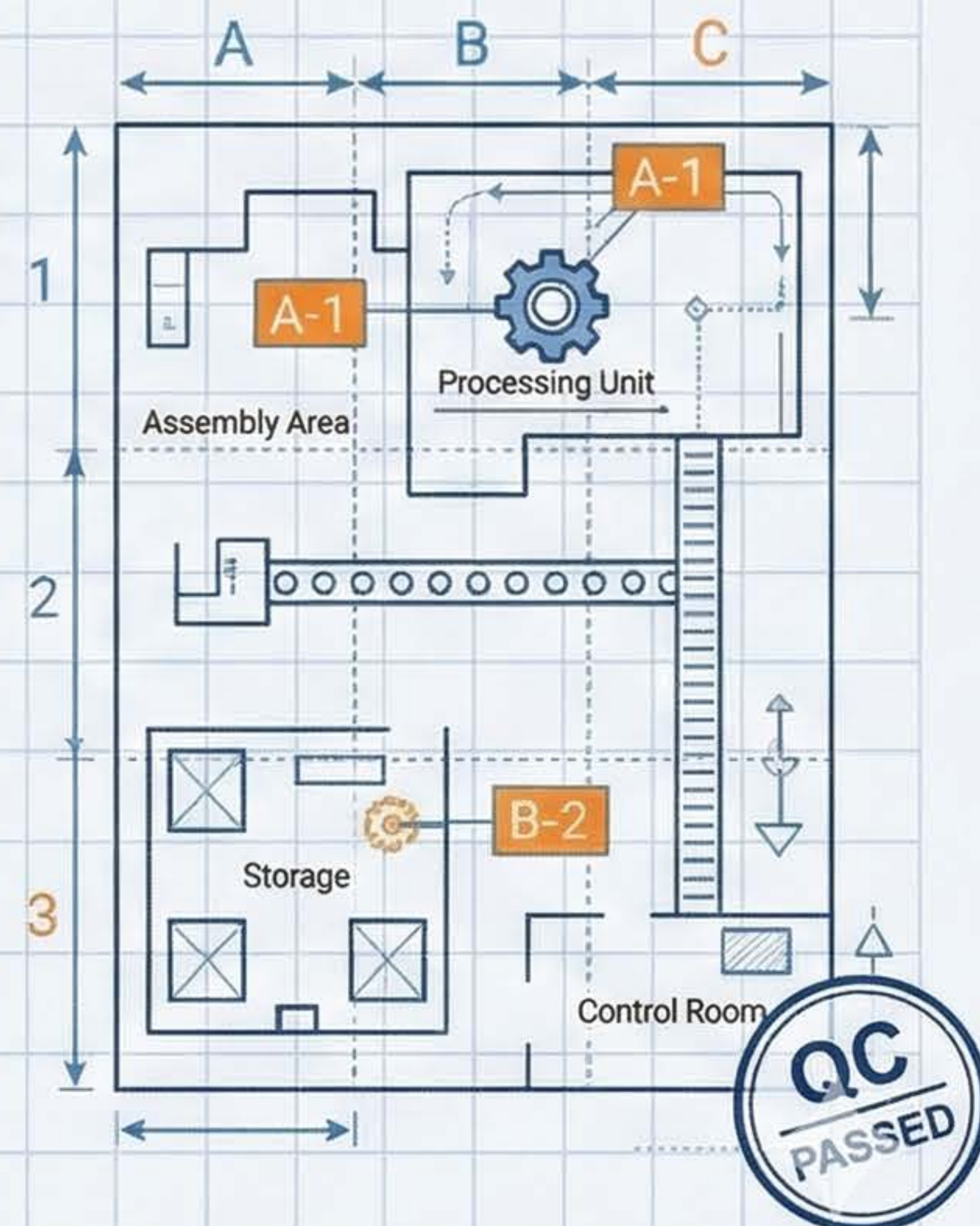


核心概念 (Core Concept)

- 資料儲存於列 (Row) 與欄 (Column) 交織的網格中。

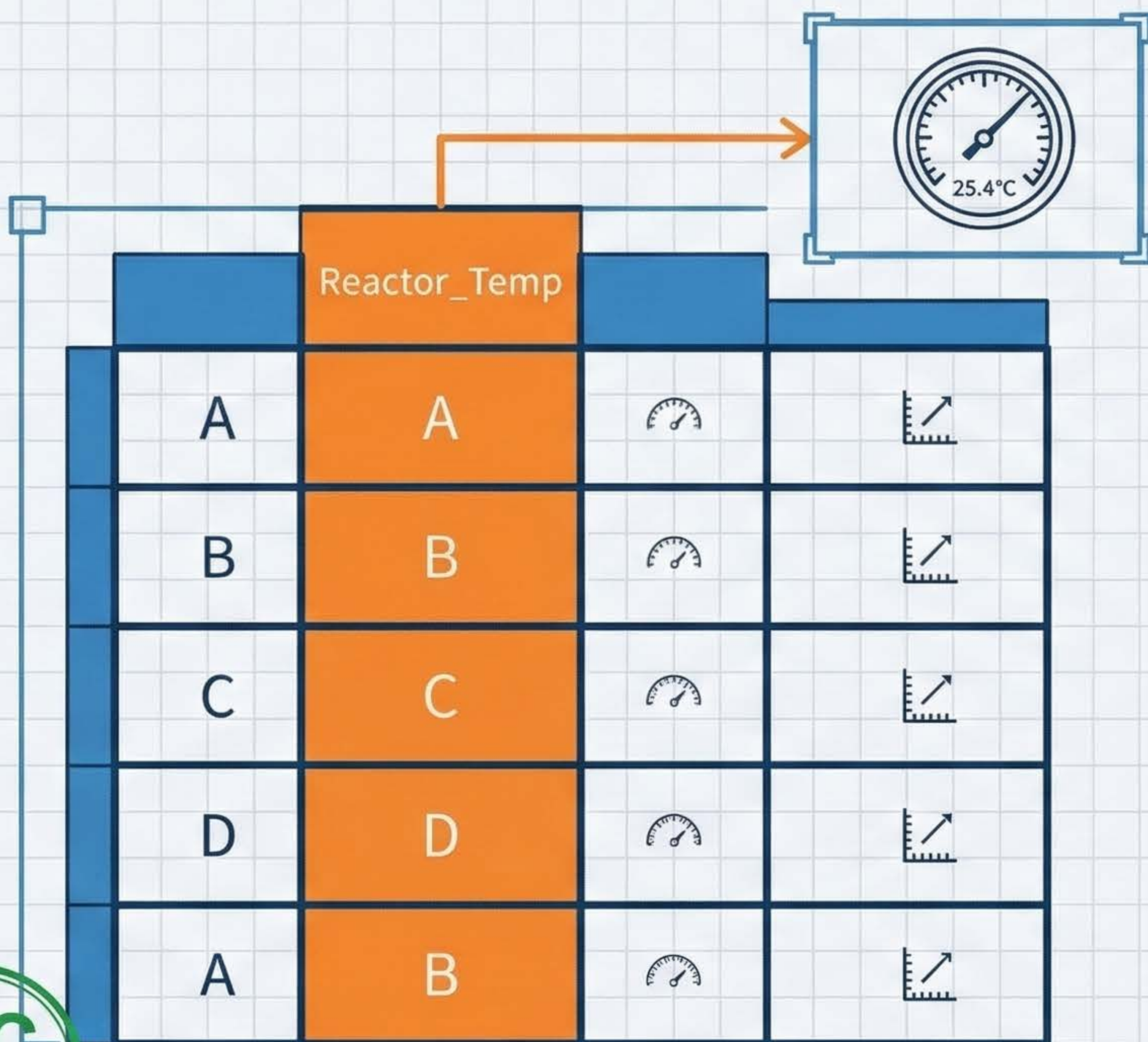
提取邏輯 (Extraction Logic)

- 我們不能像 Excel 那樣「點擊」，必須提供精確的「地址 (Address)」——即索引標籤或位置編號。





欄位選取：提取特定變數



功能 (Function)：

從 DataFrame 中提取一個 Series (單一變數)。

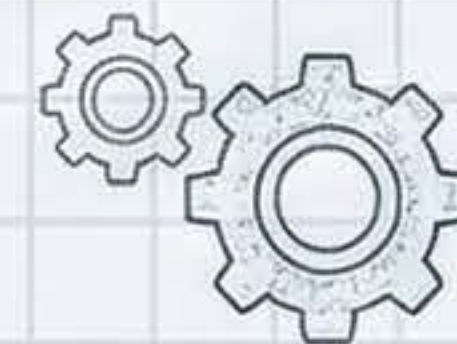
應用場景 (Scenario)：

「我只想看所有批次的『反應器溫度』數據。」

變數名稱 (Variable Name)

```
df['Reactor_Temp'] # 標準寫法 (Standard)
df.Pressure         # 屬性寫法 (Attribute Access)
```





基礎切片：範圍選取

0:5				
	D	D		
	A	B		

`df[0:5]`

語法 (Syntax) :

`df[start:stop]`

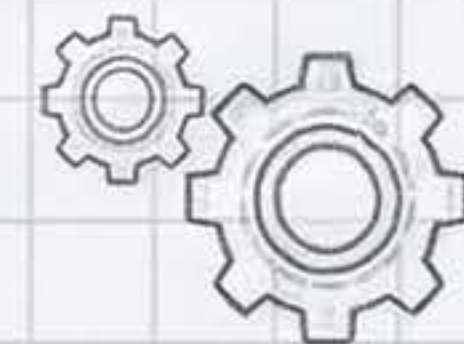
行為 (Behavior) :

選取從 `start` 到 `stop-1` 的列 (Rows)。

限制 (Limitation)

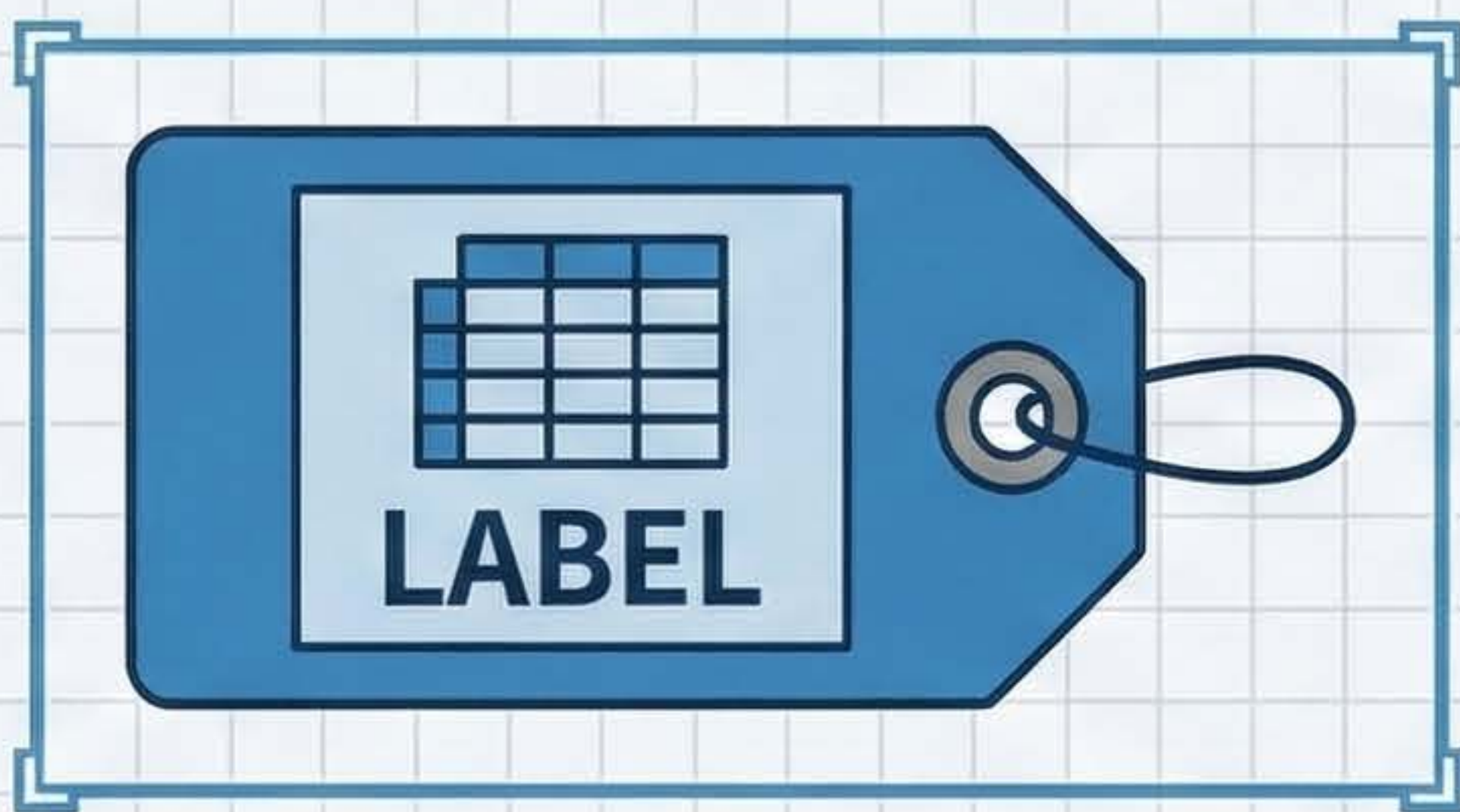
這種方式只能處理連續範圍，無法精準選取不連續的特定列（例如：第 1，3，5 筆），因此我們需要更專業的工具 (`loc/iloc`)。





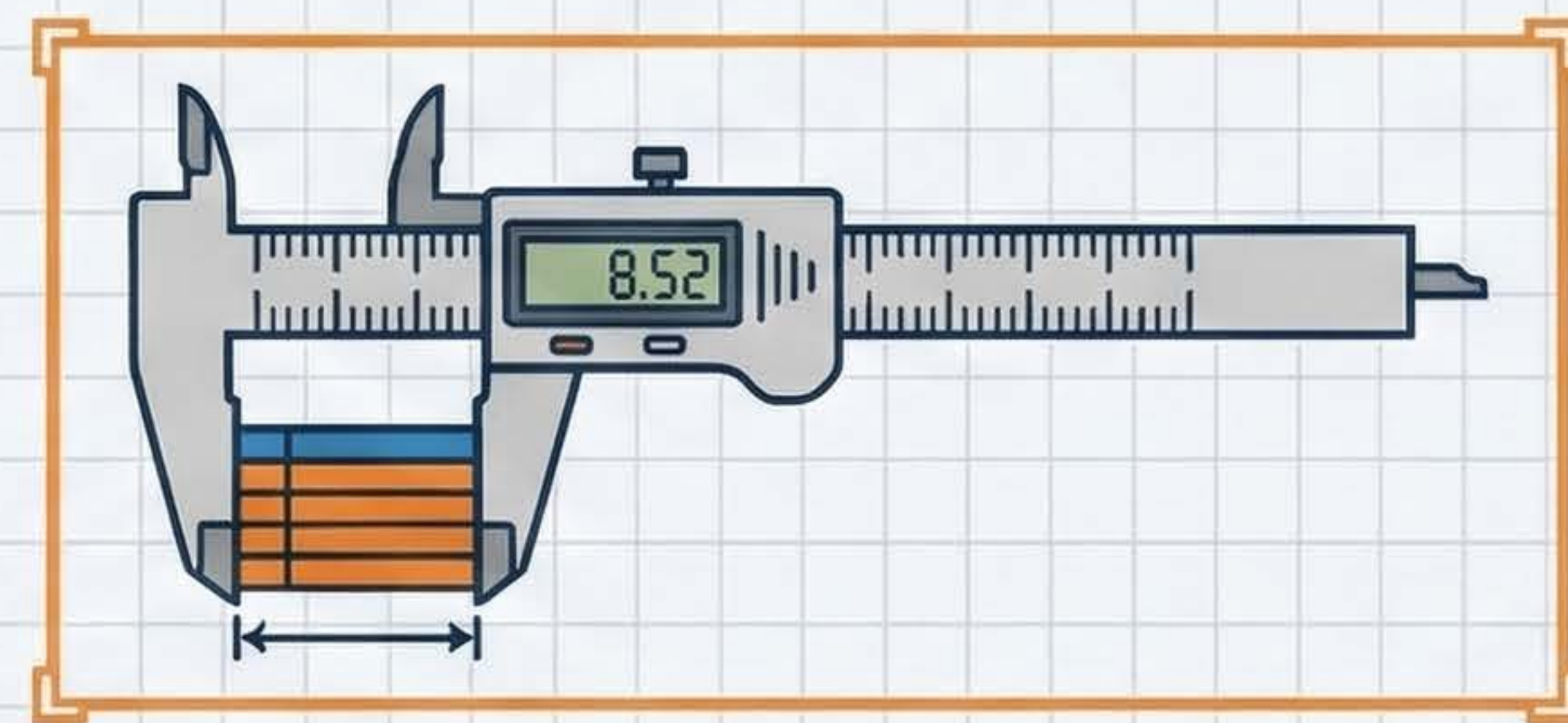
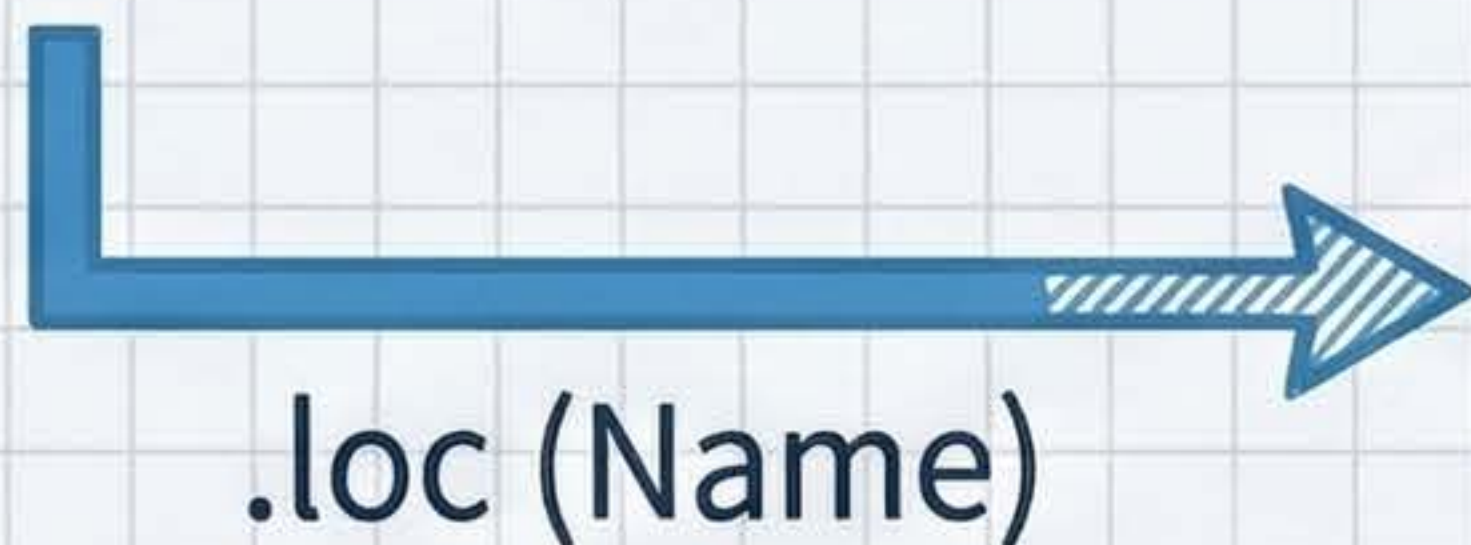
雙重導航系統：標籤 vs. 位置

Pandas 提供兩套平行的索引語言，必須明確區分以避免錯誤。



Label Logic (.loc)
依據 名稱 (Name) 查找

「找出 'Batch_A' 的生產紀錄。」



Integer Logic (.iloc)
依據 整數位置 (Position) 查找

「找出第 3 筆生產紀錄。」





.loc：基於標籤的精準定位

```
df.loc[row_label, col_label]
```

列標籤
(Index Name)

欄標籤
(Column Name)

Data Table (String Index)

	Temp	Pressure
Batch_001	85.2	...
Batch_002	...	...
Batch_003	...	...

```
df.loc['Batch_001', 'Temp']
```

→ 取得 Batch 001 的溫度

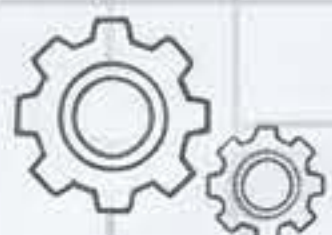
```
df.loc['Batch_001': 'Batch_003']
```

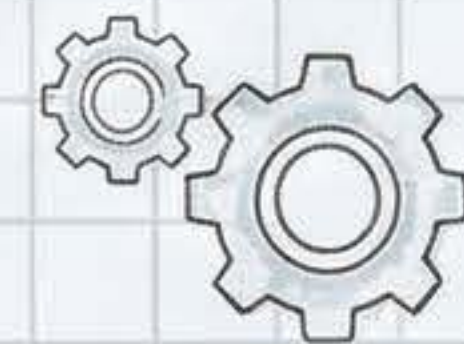
→ 範圍切片



注意 (Note)：.loc 的切片是 包含結束標籤的 (Inclusive)。

例如：['A':'C'] 會包含 'A', 'B', 和 'C'。





iloc : 基於位置的絕對座標

```
df.iloc[row_position, col_position]
```

列位置
(0, 1, 2...)

欄位置
(0, 1...)

	0	1	
X-Ray	Temp	Pressure	
0	Batch_001	85.2	...
1	Batch_002	...	...
2	Batch_003	...	...

`df.iloc[0, 1]` → 第 0 列，第 1 欄

`df.iloc[0:2]` → 前 2 列 (Row 0, Row 1)



注意 (Note) : .iloc 的切片是不包含結束位置的 (Exclusive) 。
遵循 Python 標準清單行為 (0:2 回傳 0 與 1，不含 2) 。





對決：何時使用哪一種？

情境 (Scenario)	推薦工具 (Recommended)
程式碼可讀性 (Readability) 想讓人類一眼看懂選取的是什麼數據	.loc <code>df.loc['2023-01-01']</code> (清晰) <code>df.iloc[105]</code> (神秘)
迴圈迭代與自動化 (Iteration) 使用 for 迴圈或固定結構進行批量處理	.iloc <code>for i in range(n): df.iloc[i]</code>



結論 (Key Takeaway)：

- 使用 **loc** 讓程式碼具備可讀性 (Human-readable)
- 使用 **iloc** 進行結構化運算或當索引無意義時 (Machine-readable)





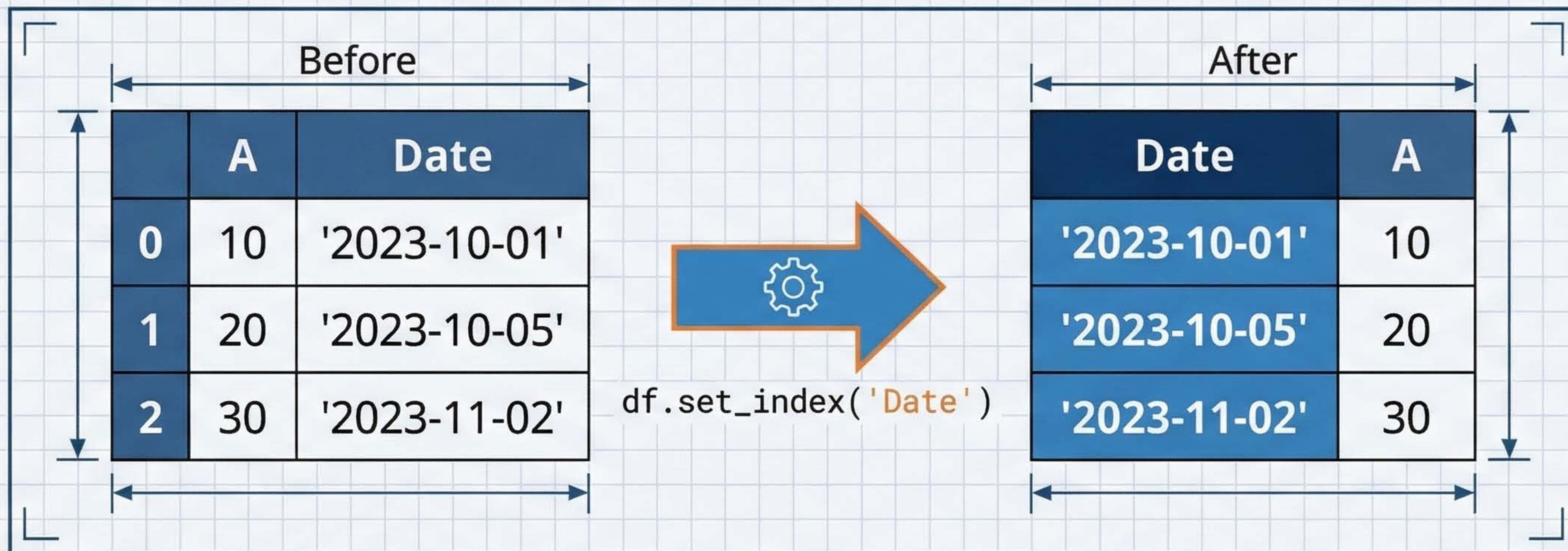
綜合應用：多維度數據提取

同時篩選特定的列 (Rows) 與特定的欄 (Columns)

```
df.loc[  
    列篩選  
    ['R-101', 'R-102'],           # Row Filter (Who)  
    欄篩選  
    ['Pressure', 'Conversion']     # Column Filter (What)  
]
```

	Temp	Pressure	Conversion	Yield
R-101				
R-102				
R-103				
R-104				

設定索引：更換門牌號碼



⚙️ **目的 (Purpose):** 將某個欄位變為索引，以啟用特定功能 (如時間序列切片)。

💻 **代碼 (Code):** `df.set_index('Date', inplace=True)`

✅ **效果 (Effect):** 現在可以使用 `df.loc['2023-10']` 快速選取該月份資料。

重置索引：恢復原廠設定

Before

Date	A
'2023-10-01'	10
'2023-10-05'	20
'2023-11-02'	30



`df.reset_index()`

After

	Date	A
0	'2023-10-01'	10
1	'2023-10-05'	20
2	'2023-11-02'	30

應用場景 (Use Case) :

1. 將索引 (Index) 變回普通欄位以便寫入 CSV。
2. 在 GroupBy 運算後將資料攤平 (Flatten)。



參數 (Parameter) :

`inplace=True` 代表直接修改原表，否則需重新賦值。

安全警示：常見陷阱與最佳實踐

Bad Practice

連鎖索引 (Chained Indexing)

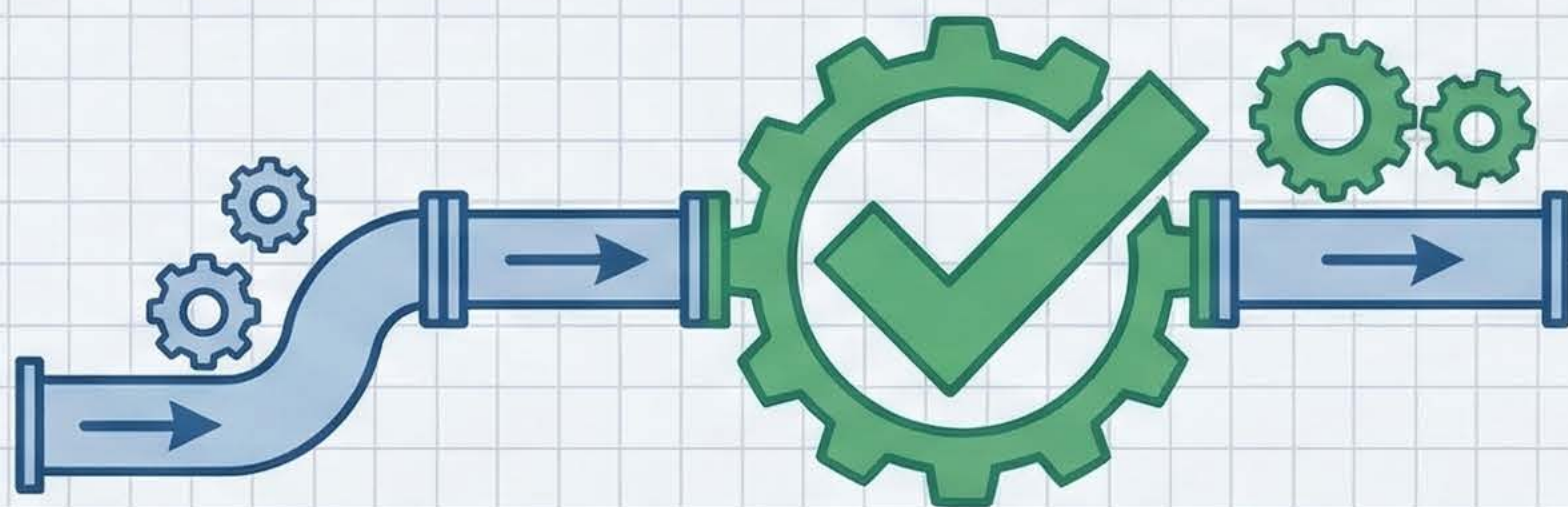
```
df['Temp'][0] = 100
```



Good Practice

明確賦值 (Explicit Assignment)

```
df.loc[0, 'Temp'] = 100
```



安全。原子操作，確保數據正確寫入。

操作總結：精準提取清單



Mastery of indexing is the prerequisite for Data Cleaning (Unit 04).